

SOFTWARE ENGINEERING
Lab 3: Architectural Justification
Problem Statement #33 — Micro-Lending & Peer Credit Risk Assessor

NAME: KAVAN S	SECTION: D
SRN: PES2UG24CS223	DATE: 08/09/2026

Architecture Selection: Microservices Architecture.

Reason 1 — Independent Scaling of the Risk Engine

The Credit Risk Assessment component performs computation-heavy scoring (analysing repayment history, peer default rates, and behavioural signals) that experiences unpredictable, bursty load whenever many borrowers apply at once. A microservices architecture lets this component be scaled or upgraded (e.g., swapping in a new scoring model) completely independently of the Loan Management, Payment Service, or User Account components, without redeploying the whole system.

Reason 2 — Fault Isolation for Financial Transactions

Because real money is disbursed and repaid through this platform, a failure in one area (for example, the notification or KYC verification component) must never bring down loan payment processing. Microservices isolate faults: if the Credit Risk Assessment service becomes slow or unavailable, borrowers can still log in, view existing loans, and make repayments, since the Payment Service and Loan Management component continue operating as separate, independently deployable services.

Security Advantage

Sensitive financial and identity data (credit history, KYC documents, payment credentials) can be confined to the specific services that need it — the Credit Risk Assessment, User Account, and Payment Service components each own their own data access and authentication boundary. This limits the blast radius of a breach: compromising the public-facing Loan Management component does not automatically expose the payment gateway credentials or credit scoring data, since those live behind separate, independently secured service boundaries and internal APIs.

Performance Benefit

Because each service is deployed and scaled independently, the platform can run multiple parallel instances of the Credit Risk Assessment component during peak application periods (e.g., month-end salary cycles) while keeping only one or two instances of lighter-weight services such as User Account management. This targeted horizontal scaling gives much better throughput per infrastructure dollar than a monolithic or purely layered design, where the entire application would need to be scaled together even though only one part is under load.